

Witness-CL: Evidence-Preserving Skill Growth, Local Auditing, and Retention Under Recurrence

Samuel Mausberg

AI-assisted research draft; author review and independent replication pending

Abstract

We study deployment-time learning from interaction feedback without an intervening offline retraining phase. An earlier evidence-preserving prototype failed when its rule class was misspecified or the environment changed without a public version. This revision supplies a schema-derived executable representation, correct replay when its class grows, immutable skill archives with recurrence-weighted recovery, and local admission tests using either real paired rollouts or one randomized observed outcome. We prove conditional replay, recovery, noninterference, and fixed-feature retention properties, while separating them from unrestricted no-forgetting claims. An information bound explains why small diffuse gains cannot generally be validated in a few episodes. A typed relational aggregate cube shares execution work across skills. Across 20-seed CPU mechanism experiments, late reward under hidden arithmetic change rises from 14.49% to 96.19%; recurrence-weighted relational recovery reaches 97.45%, versus 63.59% for reset-only learning. An integrated audited agent learns, but its audition cost lowers overall reward. Rapid switches, noisy feedback, unsupported operators, and full-rank protection remain explicit failures. The artifact contains 146 passing Python tests, 432 passing C++ cases, 29 unverified Lean theorem attempts, and uncompiled CUDA drafts. No LLM or native public-benchmark superiority, GPU speedup, or novelty of the whole combination is established.

1 Problem, scope, and existing approaches

An improving agent must turn earlier experience into better decisions on later, non-identical episodes. An archive that never deletes data does not imply that the agent selects the right evidence, preserves its meaning, or improves reward. CL-Bench evaluates shared learnable structure across stateful task streams and finds that dedicated memory mechanisms do not reliably dominate naive in-context learning (ICL) in its evaluated settings [1]. AgentCL further separates compositional reuse from misleading learning curves [2]. Raw ICL must therefore remain a primary experimental competitor, not a weak straw baseline.

Several substantial ingredients already exist. ACE

maintains evolving playbooks [3]; Voyager learns executable skills [4]; EvoTest and TTHe change the surrounding agent configuration or executable harness during testing [5, 6]. ALMA searches executable memory designs [7]. Decision-Aware Memory Cards select context by its estimated influence on actions [8]. Selecting representative synthesis examples also predates this work [9]. None of “memory as code,” “decision-relevant evidence,” or “learning at test time” is our novelty claim.

Safe policy improvement has its own strong literature. SPIBB constrains changes using baseline bootstrapping [16]; Deep SPI studies online improvement with learned representations and world models [17]. Our local audit uses real outcomes rather than assuming a learned model is a sound counterfactual oracle. Its validity still needs appropriate sampling and frozen comparisons. Sequential inference and off-policy evaluation already provide the relevant statistical foundations [10, 13]. Our recurrence and projected learning mechanisms similarly draw on tracking experts and orthogonal updates [11, 12, 14].

Research target. Can evidence-preserving executable learning improve the reward/cost/retention frontier of real continual agents relative to strong, matched-budget ICL and harness-adaptation systems? The broad benchmark question remains open here. This revision resolves concrete implementation and mathematical defects in our initial approach, tests a restricted integrated system, and specifies the conditions under which the proposed mechanisms succeed or fail.

Assumptions. The exact-elimination branch receives an immutable public input, its own chosen action, and truthful binary success. It sees no hidden regime ID or correct answer after failure. A fixed epoch’s recovery theorem assumes a deterministic true rule in the current finite class. The live statistical branch assumes randomized treatment before reward and no cross-episode treatment carryover; it does not assume that unsuccessful actions reveal unchosen rewards. General language understanding, noisy labels, and multistep interaction require separate arguments. They are not solved by calling a generated candidate set complete.

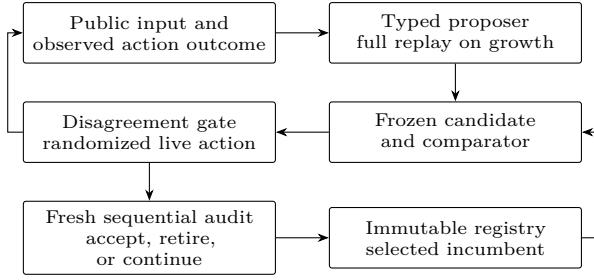


Figure 1: Implemented one-step architecture. Audit treatments are experiments, not guaranteed-safe actions. Archival retention is distinct from correct routing. No hidden task/version identifier enters the learner.

2 Architecture: grow, preserve, test, route

The system separates the proposer, the immutable skill registry, the currently selected behavior, and admission evidence. Figure 1 illustrates the implemented one-step path. An observation can improve future proposals, but a proposal must be frozen before receiving the fresh observations used to admit it. A stored skill’s identity never changes to reflect a new hypothesis.

The unaudited proposer is useful for diagnosing information reuse independently of admission cost. The audited system is a separate measured configuration, not a label attached afterward to the proposer’s higher scores. Soft expert tracking and projected parameter learning are independently implemented controls; they are not all integrated into one general-purpose LLM agent.

2.1 Executable representations grounded in public structure

The new relational intermediate representation consists of typed sum/difference plans, nullable-Boolean filters, explicit column references, and schema guards. Its grammar is generated from public column names and types. A strict JSON parser also accepts model-proposed plans without executing ambient SQL or arbitrary code. The reference interpreter and an independent read-only SQLite backend agree on empty tables, negative values, and explicit NULL behavior in the executed tests.

A plan such as “sum gross minus fee over completed, non-refunded rows, treating a missing refund flag as zero” transfers to new rows without memorizing their answers. This solves a concrete representation problem left by the arithmetic prototype. It is not a complete SQL language or a demonstrated natural-language semantic parser. Schema guards validate dependencies and types, not business meaning. Unsupported multiplication is deliberately tested and fails.

2.2 A defect in fixed-class witness compression

For a finite class H and evidence E , define

$$V(H, E) = \{h \in H : \forall e \in E, C_e(h)\}, \quad (1)$$

where $C_e(h)$ states that h ’s prediction agrees with the observed success or failure of the chosen action. Retain an observation as a witness only when it shrinks the live set. For a fixed class, replaying the witnesses gives the same set as replaying all evidence, including an empty result.

Proposition 1 (Growth requires backfill). *For any classes H, G and evidence E ,*

$$V(H \cup G, E) = V(H, E) \cup V(G, E). \quad (2)$$

A witness set sufficient for H need not be sufficient for $H \cup G$.

Proof. The equality follows by distributing membership in $H \cup G$ over the common constraint conjunction. For the counterexample, let H contain only a constant zero rule and observe a successful zero answer at $x = 1$. The witness set is empty, because nothing was eliminated. A newly proposed rule returning one at $x = 1$ passes that empty witness set and fails the original observation. $\square$

The implementation stages an enlarged class, replays the full current epoch’s raw evidence, reconstructs its witness set, and installs the result atomically. It never backfills a new rule only against an old summary. Historical incompatible epochs remain archived rather than being silently treated as stationary evidence. The raw log grows; the active sufficient witness set is not constant total memory.

2.3 Recovery without erasing old skills

When evidence becomes contradictory, the learner first tries the next bounded grammar tier. If full backfill remains inconsistent, it opens a new inference epoch, preserving old programs. This is an internal evidence epoch, not an oracle announcement of the environment’s change point. It does not distinguish all cases of drift from misspecification.

A fresh epoch uses prior

$$\pi(h) = \frac{1 - \beta}{|H|} + \frac{\beta}{|A|} \mathbf{1}\{h \in A\}, \quad (3)$$

where A is the archived subset; with an empty archive, use the uniform prior. The implementation uses $\beta = 4/5$. It predicts the maximum-mass action and eliminates inconsistent rules after exact feedback. New ambiguous explanations are marked empirical, not certified.

Theorem 2 (Conditional recurrence recovery). *Within a fixed realizable deterministic epoch, if at most $K \geq 2$ distinct actions are predicted, the number M of mistakes obeys*

$$M \leq \frac{\log(1/\pi(h^*))}{-\log(1 - 1/K)}. \quad (4)$$

For an archived truth under Eq. (3), the numerator is at most $\log(|A|/\beta)$.

Proof. Every mistaken maximum-mass action is supported by at least $1/K$ of surviving mass. Failure deletes all that mass. The true rule survives with its original positive mass $\pi(h^*)$. Thus $\pi(h^*) \leq (1 - 1/K)^M$; rearrange. The archive bound uses $\pi(h^*) \geq \beta/|A|$. $\square$

This is a weighted elimination argument, not a new general regret theorem. If a fixed final class uniquely identified the old rule, its first incorrect prediction after a hidden switch triggers contradiction. Earlier predictions remain correct until the first disagreement. That conditional one-mistake trigger does not cover noisy feedback, incomplete identification, rapid changes, or new mixed-history rules introduced by grammar expansion.

3 Admission that exploits locality

A general small improvement can be difficult to recognize. The remedy is not to use a looser significance threshold or recycle proposal data as validation. We restrict a proposed replacement to a frozen region of behavior and distinguish cheap screening from expensive fresh outcome collection.

3.1 Pointwise protection outside a patch

For pure one-step policies b, c and an observable frozen gate g , let

$$p(x) = \begin{cases} c(x) & g(x) = 1, \\ b(x) & g(x) = 0. \end{cases} \quad (5)$$

Outside the gate the complete action is unchanged. With common input/randomness, $D = R(p) - R(b) = 0$ there. Under stationary contexts,

$$\mathbb{E}D = \rho \mathbb{E}[D \mid g = 1], \quad \rho = \mathbb{P}(g = 1). \quad (6)$$

Thus positive conditional mean benefit suffices when $\rho > 0$. Protected inputs outside the patch need no statistical test. Protected subgroups inside it still need their own obligations or tests; an aggregate improvement cannot guarantee every subgroup's retention.

For an arbitrary candidate, a one-step disagreement gate $g(x) = \mathbf{1}[b(x) \neq c(x)]$ makes the patch behavior equal to c everywhere. Evaluating that gate costs both candidate predictions. It is not automatically cheap for LLMs. A syntactic applicability gate can be cheaper but must still bind the actual deployed patch.

3.2 Sequential evidence and honest roll-out accounting

For fresh paired differences $D_i \in [-1, 1]$ and predictable stakes $\lambda_i \in [0, 1]$, define

$$M_n = \prod_{i=1}^n (1 + \lambda_i D_i). \quad (7)$$

Under the null $\mathbb{E}[D_i \mid \mathcal{F}_{i-1}] \leq 0$, each factor is nonnegative with conditional mean at most one. Therefore M_n is a nonnegative supermartingale. Fixed mixtures preserve this property. The reference supplies fixed-stake tests and an additional small-stake/predictable-variance variant. These are applications of established anytime-valid inference [10, 13].

Candidate j receives $\delta_j = \delta/[j(j+1)]$ and is admitted when its mixture crosses $1/\delta_j$. A conditional Ville bound followed by a union bound controls ever accepting a null candidate by δ . The indices must be globally unique; candidates, guards, and comparators must be frozen before their audit data; an obsolete comparator cannot authorize a current replacement. These assumptions are operational requirements, not facts checked by a numerical threshold.

Proposition 3 (Zero-skipping changes cost, not evidence). *Deleting observations whose true paired difference is zero leaves a fixed-stake multiplicative audit path unchanged. It also leaves the supplied predictable variant unchanged, since its statistics update only through D_i, D_i^2 .*

Proof. A zero difference contributes factor one and changes neither running statistic. Induction over the omitted positions gives equality at every informative prefix. $\square$

At N deployment opportunities with n_g gated cases, dense evaluation uses $2N$ rollouts and screened evaluation uses $N + n_g$. The extra rollout count falls from N to n_g , approximately a $1/\rho$ factor. The *total* rollout savings are bounded by two in this accounting, and calendar waiting is unchanged. A modern variance-adaptive test may already exploit the zeros statistically. We do not claim that merely omitting them invents information or improves test power.

In a multistep environment, equality of the present action is insufficient. The valid extension runs both frozen policy states on the common observed prefix, then physically forks a simulator at the first behavioral divergence. Prefix computation and both suffixes must be charged. This extension is derived but not implemented in the present one-step harness.

3.3 One observed reward instead of two hypothetical rewards

When genuine paired forks are unavailable, independently randomize treatment $A \sim \text{Bernoulli}(e)$ before its outcome. For $e \in [p_{\min}, 1 - p_{\min}]$ and observed $R \in [0, 1]$, use

$$Z = \left(\frac{A}{e} - \frac{1-A}{1-e} \right) (R - 1/2), \quad B = \frac{1}{2p_{\min}}. \quad (8)$$

Randomization implies $\mathbb{E}Z = \mathbb{E}R(c) - \mathbb{E}R(b)$ and $|Z| \leq B$. The same test uses factors $1 + \lambda_i Z_i / B$. At $e = 1/2$, $B = 1$. The centered estimator is implemented with exact rational arithmetic and tested by enumerating treatment probabilities and reward pairs.

The integrated system uses only the reward of the executed action. It freezes a provisional candidate even before unique identification, because an uninformative incumbent can otherwise prevent discovery forever. A candidate may be retired, but its audit cannot omit unfavorable samples and resume. A replacement proposal starts after its birth observation. Audit treatments can lower individual-episode reward: safe admission is not safe exploration.

3.4 An information limit on short auditions

Proposition 4 (A standard testing lower bound). *For iid $D \in \{-1, +1\}$ with null mean zero and alternative mean $\mu > 0$, a test with $0 < a < 1 - b < 1$, null admission probability at most a , and alternative power at least $1 - b$ satisfies, under finite-horizon or appropriate integrable stopping conditions,*

$$\mathbb{E}_\mu N \geq \frac{\text{kl}(1 - b|a)}{\text{kl}((1 + \mu)/2 \| 1/2)}. \quad (9)$$

Proof. The likelihood-ratio chain rule accumulates one Bernoulli relative entropy per informative observation. Data processing to the final binary admission decision lower-bounds this total by $\text{kl}(1 - b|a)$. This is the usual change-of-measure argument [15]. $\square$

At $a = .025$, $b = .2$, and $\mu = .05$, Eq. (9) requires at least 1963.79 expected informative observations. A universally reliable 128-episode test for a five-point high-variance gain is not supported by the information. For a sparse patch with $\rho = .1$ and conditional gain .5, the same global five-point gain has a lower bound of 18.77 *gated* observations. Waiting for them still counts. The research opportunity is to construct useful low-variance local changes, not to hide the unavoidable cost of diffuse ones.

4 Retention with online parameter learning

Freezing existing modules preserves their definitions, not correct selection. For noisy or ambiguous feedback, hard deletion is inappropriate. We implement a fixed-share EXP4-style routing control over an immutable finite archive [11, 12]. With action probabilities $p_t = (1 - \gamma)q_t + \gamma/K$, selected-action loss estimates are

$$\hat{\ell}_{t,i} = \frac{\ell_t(A_t) \mathbf{1}[a_{t,i} = A_t]}{p_t(A_t)}. \quad (10)$$

After exponential updating, a share α/M restores each expert's positive weight. The standard potential analysis gives expected regret against an S -switch expert sequence bounded by

$$\frac{B_S}{\eta} + \frac{\eta K T}{2(1 - \gamma)} + \gamma T, \quad (11)$$

where

$$B_S = \log M + S \log(M/\alpha) + (T - 1 - S) \log(1/(1 - \alpha)). \quad (12)$$

This is an expectation bound for a fixed archive with nonanticipating losses, not safe exploration, a growing-expert result, or regret against the different state trajectory of a reactive environment. It can be vacuous for rapid changes.

For trainable residuals, freeze the feature extractor ϕ and prior prediction f . Let U have orthonormal columns spanning protected features and set $P = I - UU^\top$. Define

$$f_{\text{new}}(x) = f(x) + WP\phi(x). \quad (13)$$

Theorem 5 (Exact fixed-feature protected span). *If $\phi(x) \in \text{span}(U)$, then Eq. (13) equals $f(x)$ for every W . For an approximate projection $\tilde{P}$, drift is at most $\|W\| \|\tilde{P}\phi(x)\|$ in compatible norms.*

Proof. Orthogonality gives $P\phi(x) = 0$. For the approximate case, apply the operator-norm inequality to $W\tilde{P}\phi(x)$. $\square$

This is fixed-feature nullspace control, related to orthogonal-gradient methods [14], not a new neural retention theorem. It is exact in real arithmetic, not merely first-order, because the new residual is explicitly linear in projected features. The implementation uses float64 SVD and online SGD; measured drift, rather than bitwise equality, is reported. Changing the encoder, feature construction, router, or protected set changes the premise.

Proposition 6 (The capacity cost is explicit). *For isotropic features and desired linear change d^* , the least achievable new population squared error under this restriction is $\|U^\top d^*\|^2$. If the protected span has full rank d , no residual learning is possible.*

Proof. Decompose d^* into its orthogonal protected-span and complement components. Only the latter is adjustable. Isotropy makes squared prediction error equal to the squared coefficient norm. At full rank, $P = 0$. $\square$

Observable context-separated features or new frozen modules can supply additional capacity. An unobservable conflicting context cannot be resolved merely by projecting gradients. Thus exact protected behavior and learning capacity must be reported together.

5 Execution: share semantics before optimizing kernels

5.1 A shared relational aggregate cube

With F nullable Boolean fields, each row belongs to exactly one of 3^F patterns. For C numeric columns, form the integer cube

$$A[z, c] = \sum_{r: \text{pattern}(r)=z} \text{coalesce}(r_c, 0). \quad (14)$$

Each typed filter is a fixed subset of patterns. Its column sums are linear combinations of A , and a sum/difference plan selects one or two resulting columns. Disjoint partitioning and distributivity prove equivalence to the direct interpreter. The exact CPU implementation is tested against that interpreter, which is independently checked against SQLite.

Direct evaluation costs $O(HR(F+1))$ for H plans and R rows. With P_f distinct filter signatures, the cube costs

$$O(R(F+C) + P_f 3^F (F+C) + H). \quad (15)$$

The cube itself uses $O(3^F C)$ storage; cached filter sums use $O(P_f C)$ and materialized outputs use $O(H)$. This only helps for a small number of flags. The reference caps F at three and C at eight and checks integer magnitude bounds. Exponentially large cubes require fallback, not optimistic extrapolation.

5.2 Packed evidence and query-consensus kernels

The CUDA draft assigns one warp to 32 hypotheses. Integer comparisons produce a consistency ballot; bitwise intersection updates the live mask. Per-word active counts and min/max predictions feed a second per-scope reduction. Nonempty equal min/max values indicate empirical unanimity. An empty set never does. New-input consensus has a separate entry point: the predictions computed for a feedback observation cannot be reused as predictions for a future query.

The current fast path accepts int32 outputs, uint8 feedback, and uint32 masks. General relational sums use int64 and need a checked range or a future int64

specialization. No silent truncation is allowed. NVIDIA’s programming and tuning guides inform the layout, not a claim of hardware speedup [18, 19]. Tensor cores are unnecessary for this mask logic. A shared aggregate-cube GPU implementation is an explicit next kernel, not an already executed result.

Buffers must bind owner, content epoch, schema, grammar, rule ordering, query, evidence generation, precision and stream dependencies. Shape-only reuse is not a valid content cache. Small classes should stay on the CPU when launch and transfer costs dominate. Prediction generation, replay, screens, and archive traffic belong in end-to-end accounting.

6 Executed experiments

6.1 Protocol and implementation status

We use 20 seeds per mechanism setting. Arithmetic experiments have 512 episodes and three methods across seven scenarios. Relational experiments have 384 episodes and four methods across three scenarios. The integrated test has 512 episodes, two configurations and two scenarios. Together they produce 348,160 compressed per-episode records. Auditing simulations use 4,000 independent replications per condition. The fixed-share and protected-training controls have separate seeded result files. These are CPU mechanism tests, not CL-Bench, AgentCL, or LLM scores.

The environment supplies public inputs and feedback for the learner’s chosen action only. Hidden arithmetic coefficients and evaluator recipes are never passed to the learner. Relational test rows are newly generated each episode. Evaluation recipes are coded independently of grammar enumeration, although the grammar and in-grammar tasks are deliberately co-designed to test these mechanisms. That restriction must not be confused with broad semantic generalization.

The repository currently passes 146 Python tests and 432 standalone C++ cases. The 29 Lean declarations remain unverified because the toolchain is absent. CUDA sources were neither compiled nor executed. Full raw histories and the original v0.1 results are retained. Seed-level bootstrap intervals are included in the arithmetic and relational result files; table means do not establish public-benchmark significance.

6.2 Repairing the first prototype’s failure cases

Table 1 compares the actual earlier implementation against the new proposer and its archive-prior ablation on identical streams. Stationary and novel-input performance remain equal, rather than being sacrificed for change handling. Hidden change and a quadratic truth outside the initial affine class are repaired. On recur-

Scenario	v0.1	Reset	Reuse
Stationary	100.00	100.00	100.00
Unseen inputs	100.00	100.00	100.00
Hidden change	14.49	96.19	96.19
Class misspec.	10.06	100.00	100.00
Recurring	14.04	84.02	97.70
Rapid switching	14.16	17.60	22.48
Noisy feedback	14.49	47.62	65.18

Table 1: Arithmetic late-half reward (%), 20 seeds. Reset and reuse share the expanded grammar; reuse additionally weights immutable archived rules.

Scenario	Replay	Reset	Reuse
Stationary net	100.00	100.00	100.00
Recurring recipes	0.29	63.59	97.45
Unsupported op.	0.08	0.10	0.05

Table 2: Fresh-database late-half reward (%), 20 seeds. Replay knows the complete supplied grammar from the start but pools all history; it is not LLM ICL.

rence, reuse improves late reward from 84.02% to 97.70%. This is a genuine behavioral improvement, not just removal of the certificate flag. The flag is also removed because the broader assumptions no longer justify logical certification.

The limits remain visible. Rapid switches give only 22.48% late reward for the reuse variant. Five-percent feedback flips reduce it to 65.18% and can archive spurious rules. Exact elimination is not a noise-robust default. Archival outputs are checked throughout each run, but preserving a wrong or misrouted skill does not make its use correct.

6.3 Relational reuse and representation limits

For a stationary net-aggregation task, all successful symbolic variants reach 100% late reward. The full-grammar replay control is *better overall*: 95.64% versus 92.36% for staged growth. Starting with the needed representation avoids discovery cost. On returning recipes, archive weighting raises late reward from 63.59% to 97.45% without a hidden regime identifier. Full-history stationary replay fails because it combines incompatible regimes.

Unsupported multiplication defeats every variant. The reuse learner reaches only 0.05% late reward. Grammar growth is bounded, not arbitrary synthesis, and no numerical confidence score repairs a missing operation. These failures belong in the main results.

Scenario	Method	Overall	Late
Stationary	Audited	89.05	100.00
Stationary	Ungated	94.37	100.00
Recurring	Audited	75.44	84.20
Recurring	Ungated	86.92	88.83

Table 3: Integrated one-outcome experiment, reward (%). Audited and ungated configurations see the same inputs and only their executed outcomes.

Case	Cap	Pass (%)	Dense	Local
Diffuse +.05	128	3.55	124.9	124.9
Diffuse +.05	512	7.03	488.3	488.3
Diffuse +.05	2048	27.00	1773.9	1773.9
Local +.05	128	17.52	119.2	11.8
Local +.05	512	78.47	305.0	30.4
Local +.05	2048	100.00	344.3	34.5
Local null	128	0.85	127.5	12.7
Local null	512	1.12	507.9	50.7
Local null	2048	1.65	2021.1	201.8

Table 4: Mixture-audit simulations, 4,000 replications. Cap is deployment opportunities. Dense and Local are mean *extra* rollouts until crossing or cap, not total rollouts. Both audit paths have identical statistical evidence.

6.4 The integrated system pays for admission

The audited agent reaches 100% late reward on the stationary relational task, showing that proposal, exploration, audit and promotion function together. However, overall reward is 89.05%, versus 94.37% for the unaudited proposer. On recurrence, the corresponding overall values are 75.44% and 86.92%. Audit conservatism is costly; the higher proposer’s score must not be presented as a result of the audited system. On the recurring run, the audited agent averages 110.65 live audit trials, 50.65 proposed comparisons, and four promotions.

6.5 Power depends on where benefit occurs

Both positive cases in Table 4 have global mean difference .05. The diffuse case has $D = \pm 1$ with probability of a win .525. The local case has a gate of probability .1 and conditional win probability .75. At 2,048 opportunities, the diffuse case admits 27.00% of candidates, while the local case admits all simulated candidates. Local admission averages 34.46 extra rollouts, versus 344.31 for dense evaluation of the *same* local process. This is about tenfold in marginal fork cost but only about 1.82-fold in total rollout count. It is not a tenfold speedup of an agent.

The local null’s observed crossing rate is 1.65%, below the first candidate’s allocated 2.5% threshold; this fi-

Rank	New MSE	Optimum	Drift
0	0.000	0.000	0.0e+00
4	4.226	3.738	2.0e-16
12	12.958	12.246	3.8e-16
24	25.070	24.860	0.0e+00

Table 5: Online linear-residual learning in 24 dimensions. MSE is new-task mean squared error. Optimum is the protected population error floor. Drift is mean maximum absolute protected-output change across seeds.

Rows	Direct (ms)	Cube (ms)	Ratio
32	0.480	0.130	3.70
512	7.624	0.611	12.48
8192	113.285	6.710	16.88

Table 6: 64 relational plans, median CPU latency after two fixed warmup repetitions. Ratio compares the Python direct interpreter to the shared cube. It is neither a GPU kernel speedup nor an end-to-end learner measurement.

nite simulation does not prove calibration. The separate live-randomized experiment has no paired reward oracle and also finds diffuse five-point gains difficult. The predictable paired variant is unit- tested, but no favorable power claim is assigned to it without a separate study.

6.6 Protected learning exposes its capacity tradeoff

Each training run consumes 2,048 labeled examples online; no offline retraining stage is inserted. Initial new-task MSE averages 25.07. Protecting a rank-four span reduces new-task MSE to 4.23, with mean maximum protected drift approximately 2.02×10^{-16} . Unconstrained SGD fits the new target almost exactly but changes protected outputs. At rank 24, no new learning occurs. These are linear fixed-feature results, not neural-language-model retention scores.

The fixed-share control separately tests 4,096-round recurring action streams. Mean reward is 41.49%, 71.52%, and 88.34% for dwell times 32, 128, and 512. For dwell time 32 the displayed regret bound exceeds the horizon and is vacuous. This is a useful negative diagnostic, not an omitted setting.

6.7 CPU execution measurements

The factorization has exact output parity in these runs. Data change between repetitions and execution order alternates. Ratios range from 3.70 to 16.88 versus the direct Python interpreter. A vectorized database engine is a necessary future competitor. The current recurrent learner still calls the direct interpreter; its previously reported learning scores do not silently include this separately validated optimization. A GPU claim

needs prediction construction, transfers, launches, and the complete serving loop, not just a precomputed mask table.

7 What would establish or kill the broader claim?

The primary next experiment must use native stateful benchmark interfaces and real frozen models. Compare raw ICL, an established memory/playbook method, an executable harness method, and Witness-CL under equal observable feedback and full accounting of proposal, audit, retrieval and training costs. Begin with relational and codebase tasks but do not present those as coverage of forecasting, opponent modeling, or arbitrary long-horizon control. Native CL-Bench and AgentCL runs were not executed for this draft [1, 2].

Use unseen schema names and compositions, key and NULL changes, unsupported operators, correlated failures, recurrence, and latent change without public version labels. A strong parser and ICL baseline must not receive a smaller context or fewer interactions than our system. Freeze implementations, budgets, normalization, retention strata and seeds before scoring. Report seed-level paired intervals and all negative settings. A five-point aggregate normalized reward gain with no more than a one-point protected-stratum loss remains a proposed success criterion, not a prediction or an achieved result.

The main contribution is killed if gains disappear under matched budgets or stronger ICL, fail on unseen compositions, or hide protected-task regressions. The admission mechanism is killed for a workload if useful gates occur too rarely to amortize auditions. The kernel claim is killed if full prediction, replay, synchronization and transfer cost exceeds saved computation. The projected learner is inappropriate when the protected span removes all useful capacity and observable context separation is unavailable.

Resources and safety. All executed experiments need only a CPU. A GH200 can host the next local-model and CUDA validation stage, subject to measured model/context memory requirements; no GPU-hour estimate is justified before a pilot. The supplied CUDA target includes compilation and sanitizer commands, not successful results. No model service calls, API spending, external dataset submission, repository publication, or irreversible actions were performed. A real agent needs an independent permission boundary and authenticated feedback. Mean-reward admission is not an irreversible-action safety certificate.

8 Conclusion

This revision replaces three unimplemented design questions with executable, falsifiable mechanisms: schema-grounded skill growth with honest replay, recurrence-aware archival recovery, and local admission with actual outcomes. It also supplies an explicit online protected-learning construction and shared execution primitive. The improvement is substantive within the tested setting, but the outcome is not a general no-forgetting learner or a demonstrated public- benchmark winner. The restricted proofs, failures, and measured admission cost specify exactly what a larger experiment must validate.

Disclosure. Prepared as an AI-assisted research draft for Samuel Mausberg. Authorship, literature, proofs, implementation and claims require author review and independent replication. No affiliation, accepted venue, funding award or DOI is asserted. Code is MIT-licensed. The complete old draft and evidence remain in the artifact.

A Additional proof and implementation details

A.1 Fixed-class witness equivalence

Let $V_0 = H$ and $V_t = V_{t-1} \cap C_t$. If a constraint does not shrink V_{t-1} , it holds for every later subset. Induction therefore permits its omission without changing the final version space. Every retained witness removes at least one previously surviving hypothesis, so $|W_t| \leq |H| - |V_t|$. This is not a minimum cardinality witness construction. The bound counts observations, not their bits. The class-growth counterexample in Section 2 explains why this sufficiency contract must name its class explicitly.

A.2 Sketch of the fixed-share expectation bound

Interpret sharing as a Markov prior over expert paths. The probability of an S -switch comparator is at least $M^{-1}(\alpha/M)^S(1-\alpha)^{T-1-S}$, giving code length B_S . The exponential-weights potential uses $e^{-z} \leq 1 - z + z^2/2$ for $z \geq 0$ and $\log(1+u) \leq u$. It bounds estimated regret by the code length over η plus $\eta/2$ times the summed estimated second moments. Conditional expectation of a round's second moment is

$$\sum_a \frac{q_t(a)\ell_t(a)^2}{p_t(a)} \leq \frac{K}{1-\gamma}. \quad (16)$$

Comparator IPS estimates are unbiased. Mixing exploration adds at most γ loss per round. These steps give Eq. (11). None compares against an expert's counterfactual environment trajectory after different earlier actions.

A.3 Randomization and the audit filtration

The candidate and baseline functions, guard and reward normalization are fixed before their test data. Within a gated episode, assignment is independent of potential outcomes conditional on the preassignment information. Hence

$$\mathbb{E}\left[\frac{A}{e}(R-1/2)\right] = \mathbb{E}[R(c)] - 1/2, \quad (17)$$

$$\mathbb{E}\left[\frac{1-A}{1-e}(R-1/2)\right] = \mathbb{E}[R(b)] - 1/2. \quad (18)$$

Subtracting yields Eq. (8). The uniform bound B is deliberately constant. Using context-dependent normalizers can change the targeted average. After hidden drift, stationary interpretation is not automatic; a detected evidence-epoch change retires the ongoing test, but detection itself can lag.

An audit stopped for a contradicted candidate is never resumed. Admissions remain a subset of crossings of its full, uncensored potential test path. A newly proposed candidate excludes the observation that produced it via its strict birth cut. Unlimited restarts without new global error indices would invalidate the claim. The reference allocator is single-process; durable distributed indexing is not implemented.

A.4 Formalization and execution boundaries

`Core.lean` contains 13 earlier theorem attempts. The 16 additions in `Refinement.lean` cover replay under class growth, its explicit counterexample, guarded patch noninterference, immutable archive outputs, an integer residual identity, and neutral-factor algebra. They use no placeholders or added axioms, but `lake build` did not execute because `lake` is unavailable. There is no kernel-checked Lean result to cite.

Probability, real matrix algebra, the sample lower bound, and compiler refinement remain handwritten arguments. Python and C++ test results do not prove the CUDA program. The C++ oracle has 432 random/tail/idempotence cases. The unexecuted CUDA runner adds empty-mask and extreme-int sentinel cases. Multistream, graph replay, changing-content and buffer-reuse validation are still required before production.

A.5 Reproduction

Install the test/analysis extras; run `make test`, `make refine`, `make cpp-check`, and `make paper`. Tables are regenerated from recorded JSON. Complete compressed episode records, raw timing repetitions, and the original draft and data are retained. CUDA checks require a CUDA device and compiler.

References

- [1] Parth Asawa, Christopher M. Glaze, Gabriel Orlanski, Ramya Ramakrishnan, Benji Xu, Asim Biswal, Vincent Sunn Chen, Frederic Sala, Matei Zaharia, Joseph E. Gonzalez. Continual Learning Bench: Evaluating Frontier AI Systems in Real-World Stateful Environments. 2026. <https://arxiv.org/abs/2606.05661>.
- [2] Yiheng Shu, Bernal Jiménez Gutiérrez, Saisri Padmaja Jonnalagedda, Yuguang Yao, Huan Sun, Yu Su. AgentCL: Toward Rigorous Evaluation of Continual Learning in Language Agents. 2026. <https://arxiv.org/abs/2606.02461>.
- [3] Qizheng Zhang, et al.. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models. 2025. <https://arxiv.org/abs/2510.04618>.
- [4] Guanzhi Wang, et al.. Voyager: An Open-Ended Embodied Agent with Large Language Models. 2023. <https://arxiv.org/abs/2305.16291>.
- [5] Yufei He, et al.. EvoTest: Evolutionary Test-Time Learning for Self-Improving Agentic Systems. 2025. <https://arxiv.org/abs/2510.13220>.
- [6] Jun Nie, et al.. TTHe: Test-Time Harness Evolution. 2026. <https://arxiv.org/abs/2607.08124>.
- [7] Yiming Xiong, Shengran Hu, Jeff Clune. Learning to Continually Learn via Meta-learning Agentic Memory Designs. 2026. <https://arxiv.org/abs/2602.07755>.
- [8] Xinyu Guan, Qianyang Zhao, Yuming Deng. Decision-Aware Memory Cards: Counterfactual-Inspired Context Selection and Compression for Tool-Using LLM Agents. 2026. Version 3, September 4. <https://arxiv.org/abs/2606.08151>.
- [9] Yewen Pu, Zachery Miranda, Armando Solar-Lezama, Leslie Kaelbling. Selecting Representative Examples for Program Synthesis. In *Proceedings of the 35th International Conference on Machine Learning*, 2018. <https://proceedings.mlr.press/v80/pu18b.html>.
- [10] Steven R. Howard, Aaditya Ramdas, Jon McAuliffe, Jasjeet Sekhon. Time-uniform, Nonparametric, Nonasymptotic Confidence Sequences. *The Annals of Statistics*, 49(2):1055–1080, 2021. <https://arxiv.org/abs/1810.08240>.
- [11] Mark Herbster, Manfred K. Warmuth. Tracking the Best Expert. *Machine Learning*, 32:151–178, 1998. <https://doi.org/10.1023/A:1007424614876>.
- [12] Peter Auer. Using Confidence Bounds for Exploitation-Exploration Trade-offs. *Journal of Machine Learning Research*, 3:397–422, 2002. <https://jmlr.org/papers/v3/auer02a.html>.
- [13] Ian Waudby-Smith, Lili Wu, Aaditya Ramdas, Nikos Karampatziakis, Paul Mineiro. Anytime-valid Off-policy Inference for Contextual Bandits. 2022. <https://arxiv.org/abs/2210.10768>.
- [14] Mehrdad Farajtabar, Navid Azizan, Alex Mott, Ang Li. Orthogonal Gradient Descent for Continual Learning. In *International Conference on Artificial Intelligence and Statistics*, 2020. <https://proceedings.mlr.press/v108/farajtabar20a.html>.
- [15] Emilie Kaufmann, Olivier Cappé, Aurélien Garivier. On the Complexity of Best-Arm Identification in Multi-Armed Bandit Models. *Journal of Machine Learning Research*, 17(1):1–42, 2016. <https://jmlr.org/papers/v17/kaufman16a.html>.
- [16] Romain Laroche, Paul Trichelair, Remi Tachet Des Combes. Safe Policy Improvement with Baseline Bootstrapping. In *International Conference on Machine Learning*, 2019. <https://proceedings.mlr.press/v97/laroche19a.html>.
- [17] Florent Delgrange, Raphael Avalos, Willem Röpke. Deep SPI: Safe Policy Improvement via World Models. In *International Conference on Learning Representations*, 2026. https://proceedings.iclr.cc/paper_files/paper/2026/file/0212b0dff7b4aa3bd8b8cc4ce24fd384-Paper-Conference.pdf.
- [18] NVIDIA. CUDA Programming Guide. 2026. Consulted September 8; kernel drafts were not compiled. <https://docs.nvidia.com/cuda/cuda-programming-guide/index.html>.
- [19] NVIDIA. NVIDIA Hopper Tuning Guide. 2026. Consulted September 8. <https://docs.nvidia.com/cuda/hopper-tuning-guide/index.html>.